Light

GCSE Computer Science - Programming Testing and Robustness (Set 3)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • AQA 8525 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Programming Testing And Robustness

Multiple-choice topic test for **GCSE Computer Science (AQA 8525)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

What does it mean for a program to be 'robust'?

- ○ **A.** The program uses the latest programming language
- ○ **B.** The program can handle unexpected inputs and errors without crashing
- ○ **C.** The program has many lines of code
- ○ **D.** The program runs very quickly

Q2 (1 mark)

Why should programmers test with data they did not expect users to enter?

- ○ **A.** To make the testing process take longer
- ○ **B.** Because unexpected data will never actually be entered
- ○ **C.** To ensure the program handles unusual or invalid inputs gracefully
- ○ **D.** To prove the program is impossible to break

Q3 (1 mark)

Which error message indicates a syntax error?

- ○ **A.** The program calculated the wrong total
- ○ **B.** The loop ran one too many times
- ○ **C.** The average is higher than expected
- ○ **D.** Unexpected end of input - missing closing bracket

Q4 (1 mark)

A program accepts percentage values from 0 to 100. Which value is boundary test data?

- ○ **A.** 50
- ○ **B.** 75
- ○ **C.** 25
- ○ **D.** 101

Q5 (1 mark)

What does a type check validate?

- ○ **A.** That the data is the expected kind (such as integer, string, or real)
- ○ **B.** That the data is within an acceptable numeric range
- ○ **C.** That something has been entered in the field
- ○ **D.** That the username and password match

Q6 (1 mark)

Which situation most clearly indicates a logic error?

- ○ **A.** The code will not run because of a missing colon
- ○ **B.** The program runs but the discount calculation gives £15 instead of £10
- ○ **C.** The programming environment shows a red underline
- ○ **D.** A file cannot be found when the program starts

Q7 (1 mark)

Why is it important that authentication checks BOTH username AND password?

- ○ **A.** Because checking only one would let attackers gain access by knowing just one credential
- ○ **B.** To make the program run faster

- ○ **C.** To save memory space
- ○ **D.** Because usernames are always the same as passwords

Q8 (1 mark)

A thermostat accepts temperature settings from 10 to 30 degrees. Which validation is most appropriate?

- ○ **A.** Check if the temperature contains letters
- ○ **B.** Check if the temperature is between 10 and 30 inclusive
- ○ **C.** Check if exactly 2 digits were entered
- ○ **D.** Check if the temperature is a prime number

Q9 (1 mark)

What makes test data 'normal' or 'typical'?

- ○ **A.** It represents everyday values that users would commonly enter
- ○ **B.** It deliberately breaks the program
- ○ **C.** It is always exactly at the boundary values
- ○ **D.** It is always in the wrong format

Q10 (1 mark)

A form requires users to enter their phone number. What validation ensures this field is not left empty?

- ○ **A.** Range check
- ○ **B.** Format check
- ○ **C.** Length check
- ○ **D.** Presence check

Q11 (1 mark)

What is the primary purpose of debugging?

- ○ **A.** To add new features to a program
- ○ **B.** To find and fix errors in a program
- ○ **C.** To make a program run on different computers
- ○ **D.** To write documentation for the program

Q12 (1 mark)

Which characteristic best describes erroneous test data?

- ○ **A.** It falls within the acceptable range of values
- ○ **B.** It represents typical user input
- ○ **C.** It is clearly invalid and should be rejected by the program
- ○ **D.** It is at the boundary between valid and invalid

Q13 (2 marks)

What is output when the user enters 7?

```
quantity = INPUT("Enter quantity: ")
IF quantity >= 1 AND quantity <= 5 THEN
    OUTPUT "Order placed"
ELSE
    OUTPUT "Maximum 5 items per order"
ENDIF
```

- ○ **A.** Order placed
- ○ **B.** 7
- ○ **C.** Error
- ○ **D.** Maximum 5 items per order

Q14 (2 marks)

This code should output "Even" for even numbers. What does it output for input 6?

```
num = INPUT("Enter number: ")
IF num MOD 2 = 1 THEN
    OUTPUT "Even"
ELSE
    OUTPUT "Odd"
ENDIF
```

- ○ **A.** Odd
- ○ **B.** Even
- ○ **C.** 6

- D. 1

Q15 (2 marks)

Identify the error in this code:

```
name = INPUT("Enter name: ")
IF name = "" THEN
    OUTPUT "Name required"
ELSIF name.length > 50 THEN
    OUTPUT "Name too long"
ENDIF
```

- A. Logic error - wrong comparison
- B. Syntax error - ELSIF should be ELSE IF in AQA pseudocode
- C. No error - code is correct
- D. Runtime error - name might be empty

Q16 (2 marks)

What is output when username "admin" and password "admin" are entered?

```
validUser = "admin"
validPass = "admin123"

user = INPUT("Username: ")
pass = INPUT("Password: ")

IF user = validUser THEN
    IF pass = validPass THEN
        OUTPUT "Welcome"
    ELSE
        OUTPUT "Wrong password"
```

```
    ENDIF
ELSE
    OUTPUT "Unknown user"
ENDIF
```

- ○ **A.** Welcome
- ○ **B.** Unknown user
- ○ **C.** Wrong password
- ○ **D.** admin

Q17 (2 marks)

A program accepts quantities from 1 to 999. A student tests with 1, 500, 999, 0, 1000, "five". Which classification is correct?

- ○ **A.** All values are normal test data
- ○ **B.** 1 and 999 are boundary; 500 is normal; 0, 1000, "five" are erroneous
- ○ **C.** All values are erroneous test data
- ○ **D.** Only 500 is valid test data

Q18 (2 marks)

What is output when "AB" is entered?

```
code = INPUT("Enter product code: ")
IF LENGTH(code) = 5 THEN
    OUTPUT "Valid code"
ELSE
```

```
    OUTPUT "Code must be 5 characters"
ENDIF
```

- ○ **A.** Valid code
- ○ **B.** AB
- ○ **C.** 2
- ○ **D.** Code must be 5 characters

Q19 (2 marks)

This code should give a 10% discount for orders over £50. Identify the error:

```
total = 75
IF total > 50 THEN
    discount = total * 10
    final = total - discount
ENDIF
OUTPUT final
```

- ○ **A.** Logic error - should be * 0.1 for 10%, not * 10
- ○ **B.** Syntax error - missing ELSE
- ○ **C.** No error - the code is correct
- ○ **D.** Runtime error - final might not be defined

Q20 (2 marks)

What is output when 12 is entered?

```
month = INPUT("Enter month (1-12): ")
IF month < 1 THEN
    OUTPUT "Too low"
ELSE IF month > 12 THEN
    OUTPUT "Too high"
ELSE
    OUTPUT "Valid month"
ENDIF
```

- ○ **A.** Too low
- ○ **B.** Too high
- ○ **C.** Valid month
- ○ **D.** 12

Q21 (2 marks)

What correction fixes the syntax error in this code?

```
WHILE count < 10
    OUTPUT count
    count = count + 1
WHILE END
```

- ○ **A.** Change WHILE count < 10 to WHILE (count < 10)
- ○ **B.** Change count = count + 1 to count <- count + 1
- ○ **C.** Add THEN after the condition
- ○ **D.** Change WHILE END to ENDWHILE

Q22 (2 marks)

A password must be between 8 and 20 characters. Which passwords test the boundaries correctly?

- ○ **A.** "pass" (4 chars), "password123" (11 chars)
- ○ **B.** "passwor" (7 chars), "password" (8 chars), "12345678901234567890" (20 chars), "123456789012345678901" (21 chars)
- ○ **C.** "a" (1 char), "ab" (2 chars)
- ○ **D.** "test" (4 chars), "testing" (7 chars)

Q23 (2 marks)

What is the key difference between finding a syntax error and finding a logic error?

- ○ **A.** Syntax errors are more serious than logic errors
- ○ **B.** Syntax errors are found by the interpreter/compiler before running; logic errors are found through testing outputs
- ○ **C.** Logic errors prevent programs from running; syntax errors just give wrong answers
- ○ **D.** There is no difference - they are the same thing

Q24 (3 marks)

This code validates a PIN. What is output when "1234" is entered?

```
pin = INPUT("Enter PIN: ")
errors = []
```

```
IF LENGTH(pin) != 4 THEN
    errors.append("PIN must be 4 digits")
ENDIF

FOR i <- 0 TO LENGTH(pin) - 1
    IF pin[i] < "0" OR pin[i] > "9" THEN
        errors.append("PIN must contain only numbers")
    ENDIF
ENDFOR

IF LENGTH(errors) = 0 THEN
    OUTPUT "PIN accepted"
ELSE
    FOR e IN errors
        OUTPUT e
    ENDFOR
ENDIF
```

- ○ **A.** PIN must be 4 digits
- ○ **B.** PIN must contain only numbers
- ○ **C.** PIN accepted
- ○ **D.** PIN must be 4 digits and PIN must contain only numbers

Q25 (3 marks)

This login system has a security flaw. Identify the problem.

```
storedUser = "manager"
storedPass = "secure2024"
attempts = 0

REPEAT
    user = INPUT("Username: ")
    pass = INPUT("Password: ")
    attempts = attempts + 1

    IF user = storedUser AND pass = storedPass THEN
```

```
        OUTPUT "Access granted"
    ELSE
        OUTPUT "Access denied. Try again."
    ENDIF
UNTIL user = storedUser AND pass = storedPass
```

- ○ **A.** There is no limit on login attempts, allowing unlimited password guessing
- ○ **B.** The password is stored in plain text
- ○ **C.** The username check should use OR instead of AND
- ○ **D.** The loop should be a FOR loop

Q26 (3 marks)

This code should find the smallest number in an array but outputs 0. Identify the bug.

```
numbers = [45, 23, 67, 12, 89]
smallest = 0

FOR i <- 0 TO 4
    IF numbers[i] < smallest THEN
        smallest = numbers[i]
    ENDIF
ENDFOR

OUTPUT smallest
```

- ○ **A.** The loop should start at 1
- ○ **B.** The comparison should be > not <
- ○ **C.** smallest should be initialised to the first array element or a very large value, not 0

- ○ **D.** The array indices are wrong

Q27 (3 marks)

A game rating system accepts ages from 3 to 100. Design comprehensive test data for this validation.

- ○ **A.** Normal: 25, 50; Boundary: 2, 3, 100, 101; Erroneous: -5, "ten", 150
- ○ **B.** Test only age 50
- ○ **C.** Test ages 1, 2, 3, 4, 5
- ○ **D.** Normal: 3, 100; Boundary: 50; Erroneous: 25

Q28 (3 marks)

Analyse this code and identify all errors.

```
1: average = 0
2: count = 0
3: total = 0
4: FOR num IN numbers
5:     total = total + num
6:     count = count + 1
7: ENDFOR
8: average = total / count
9: OUTPUT "Average: " average
```

- ○ **A.** One syntax error: missing concatenation operator on line 9
- ○ **B.** One logic error: average calculated before loop
- ○ **C.** No errors - the code is correct

○ **D.** Two errors: potential division by zero if array empty (logic), and missing + in OUTPUT (syntax)

Q29 (3 marks)

A library system validates book ISBN codes (must be exactly 13 digits). Analyse which validation is most complete.

○ **A.** IF LENGTH(isbn) = 13 THEN valid

○ **B.** IF isbn contains only digits THEN valid

○ **C.** IF LENGTH(isbn) = 13 AND isbn contains only digits THEN valid

○ **D.** IF isbn is not empty THEN valid

Q30 (3 marks)

A programmer tested a discount calculator with values 50, 100, and 150 (all giving 10% discount for orders over £25). The tests all passed. Why might bugs still exist?

○ **A.** The tests used too many values

○ **B.** The test data was all normal data - no boundary or erroneous testing was done

○ **C.** The discount percentage is too low

○ **D.** Testing can never find any bugs

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.